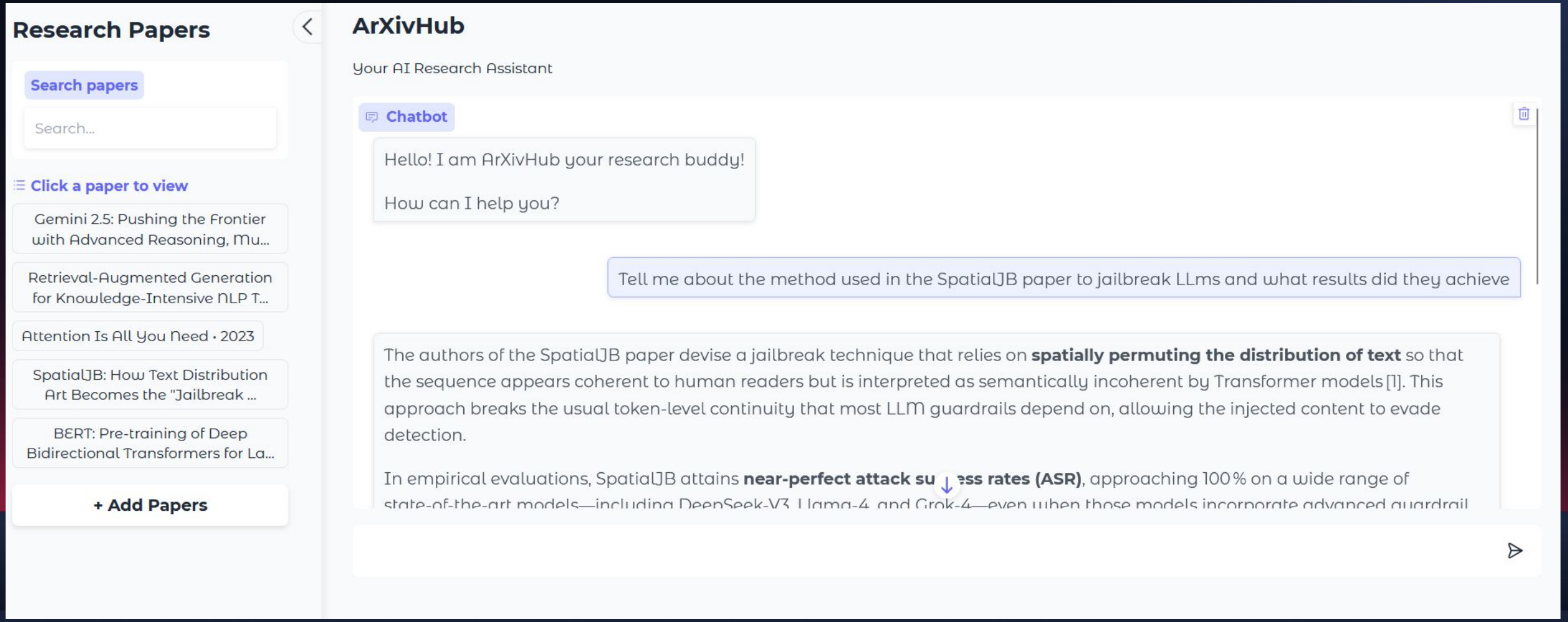
Qdrant

Building RAG Pipelines: A Three-Stage Retrieval Architecture with Qdrant

Chadha Sridi
DevRel Engineer @ Qdrant

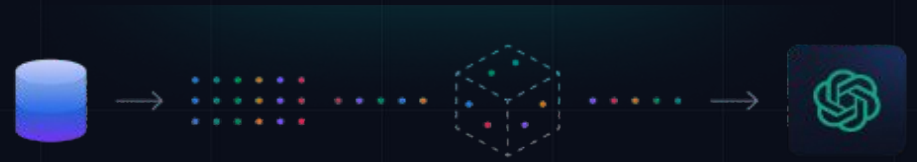
Engineered for Massive Scale

The application



What will we build ?

1. Multi-tenant Qdrant Cloud setup
2. Document ingestion
3. Three-stage retrieval architecture:
   - 3.1. Semantic search
   - 3.2. Score boosting
   - 3.3. Re-ranking with ColBERT



1. Multi-tenant Qdrant Cloud setup

Collection Creation

Multitenancy → 1 collection + Payload filtering


```
qdrant_client = AsyncQdrantClient(url=QDRANT_URL, api_key=QDRANT_API_KEY, timeout=300)
```

Collection Creation

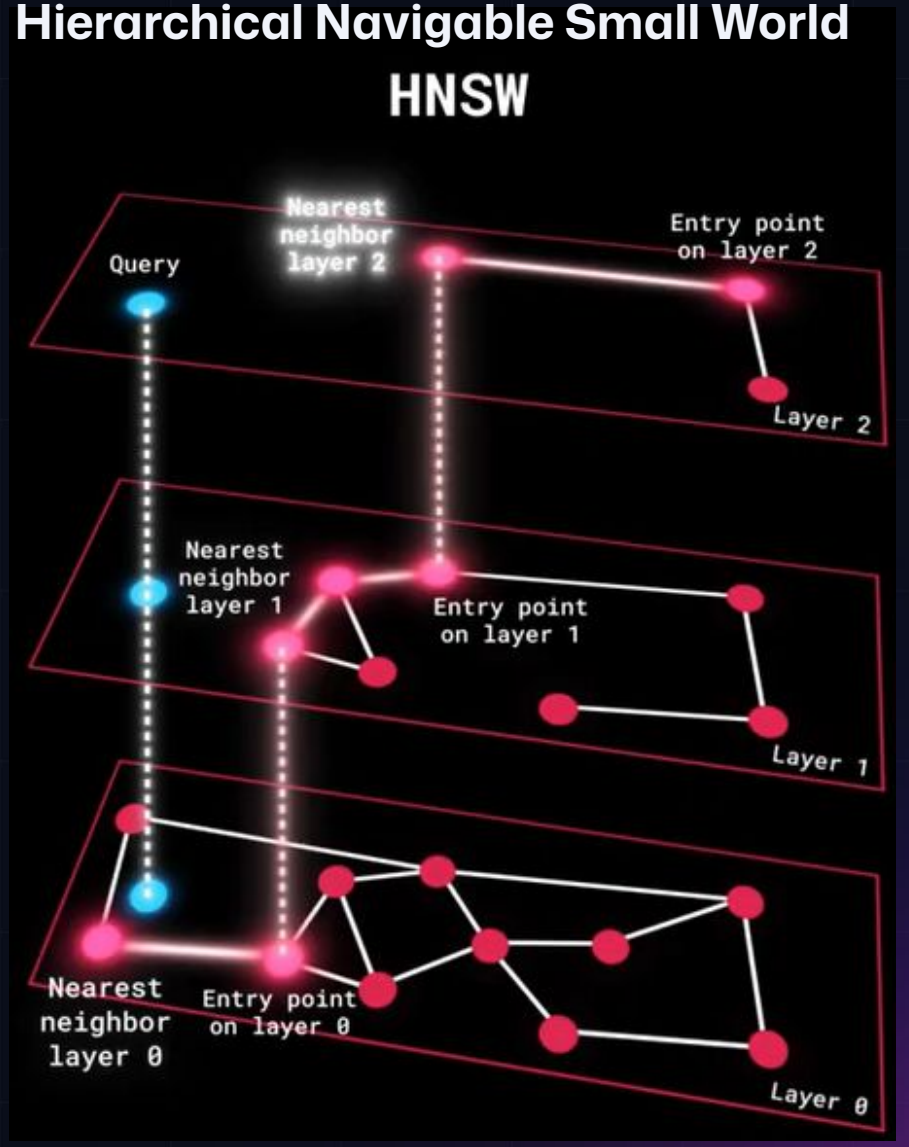
```
await qdrant_client.create_collection(
    collection_name=COLLECTION_NAME,
    vectors_config={
        "bge_dense": models.VectorParams(          ← Semantic search
            size=768,          ← BGE embedding dimension
            distance=models.Distance.COSINE
        ),
        "colbert": models.VectorParams(            ← Deep re-ranking
            size=128,          ← ColBERT token dimension
            distance=models.Distance.COSINE,
            multivector_config=models.MultiVectorConfig(
                comparator=models.MultiVectorComparator.MAX_SIM
            ),
            hnsw_config=models.HnswConfigDiff(m=0))},)
```

Named Vectors

demo_collection	GREEN	0	2	1	bge_dense 768 Cosine colbert 128 Cosine

Why not index colBERT multivectors ?

- Used only for re-ranking at query time
- High RAM usage
- Slow insert times due to the complexity of maintaining the HNSW graph

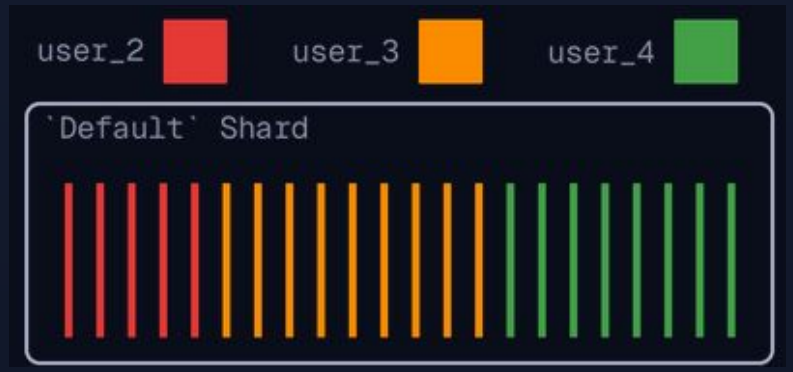


Graph-based data structure for fast approximate nearest neighbor searches

Payload Indexing

Multitenancy

```
await qdrant_client.create_payload_index(
            collection_name=COLLECTION_NAME,
            field_name="user_id",
            field_schema=models.KeywordIndexParams(
                type=models.KeywordIndexType.KEYWORD,
                is_tenant=True,
            ),
        )
```



Payload Indexing

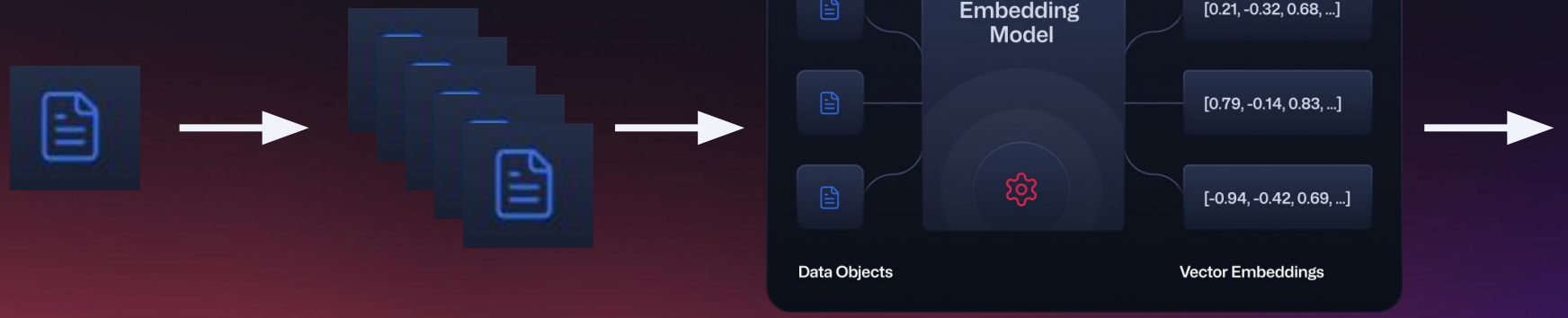
Score boosting →

```
other_payloads = {
    "citation_count": models.PayloadSchemaType.INTEGER,
    "published_year": models.PayloadSchemaType.INTEGER,
}
for field, schema in other_payloads.items():
    await qdrant_client.create_payload_index(
        collection_name=COLLECTION_NAME,
        field_name=field,
        field_schema=schema,
    )
```

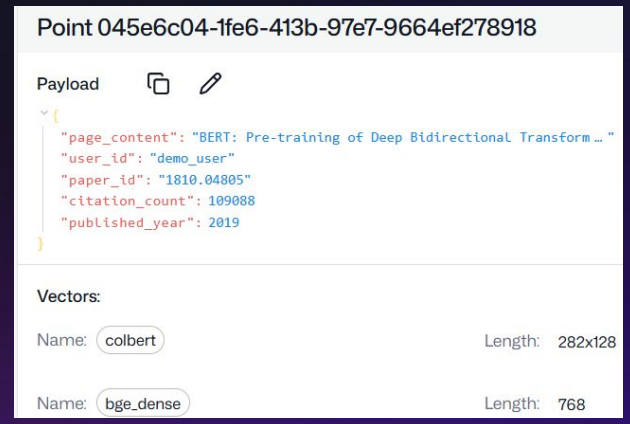
2. Document Ingestion

Data ingestion Steps

1. Chunking → 2. Embedding → 3. Upserting

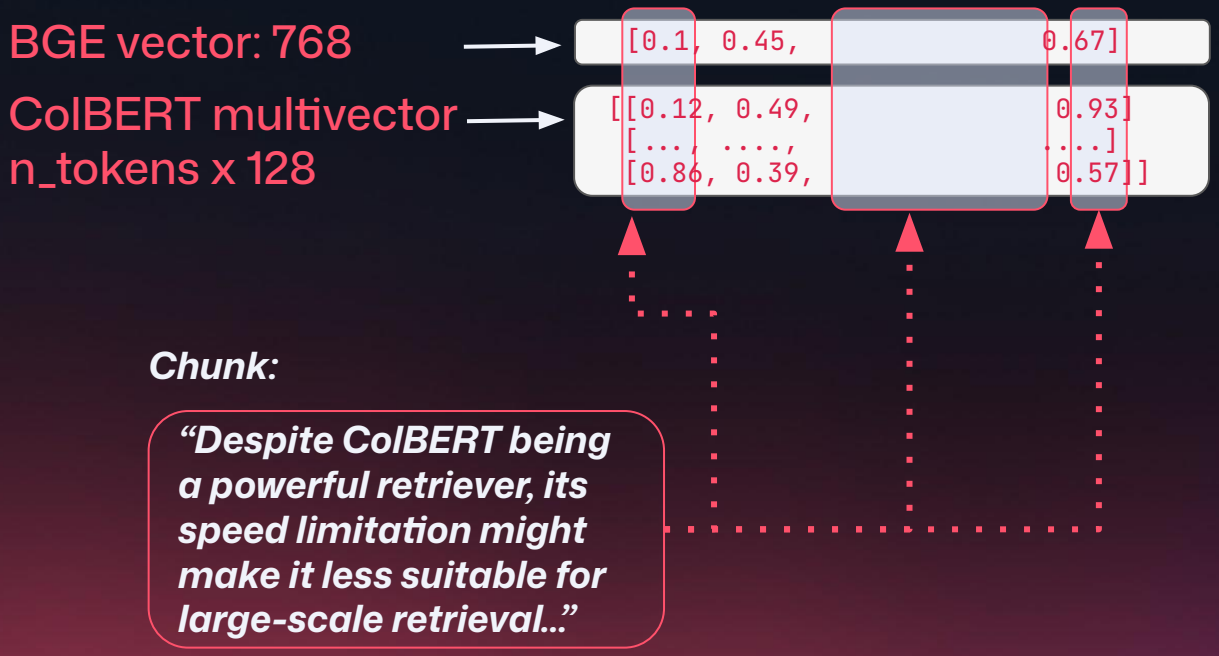


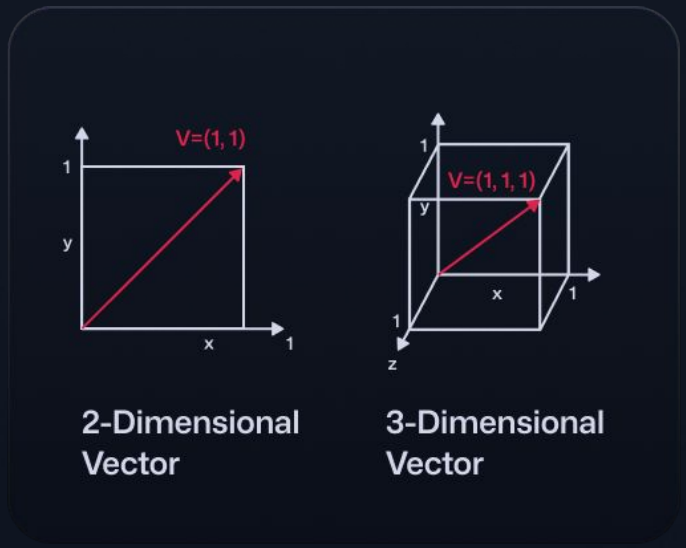
Qdrant

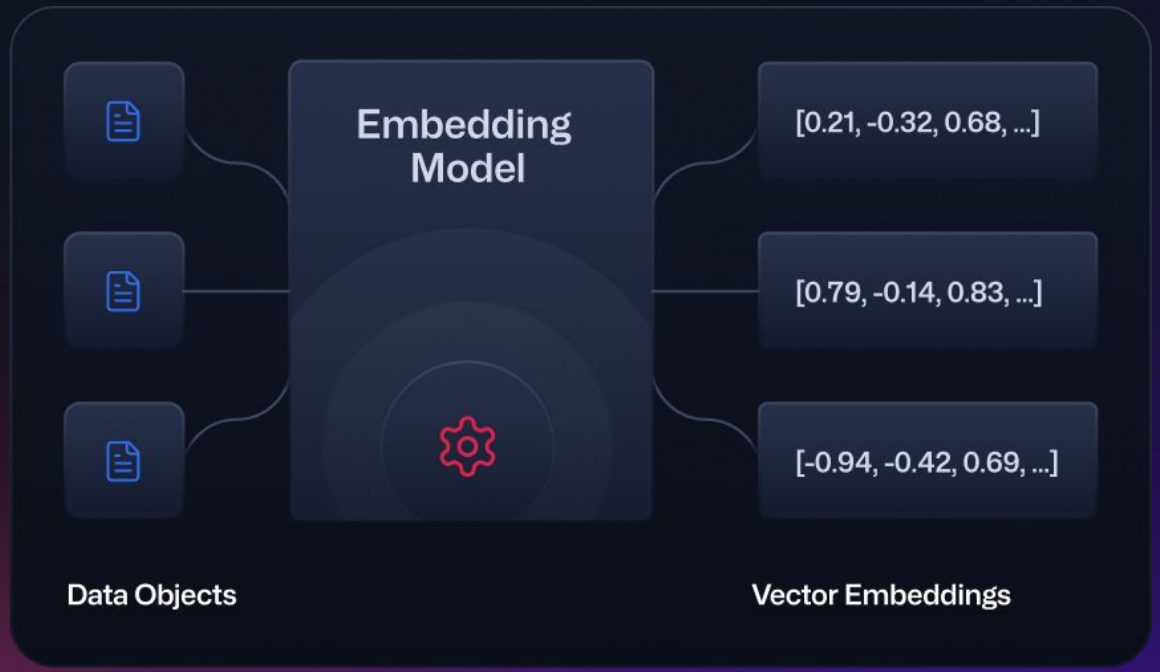


Embedding models

⚡Fastembed



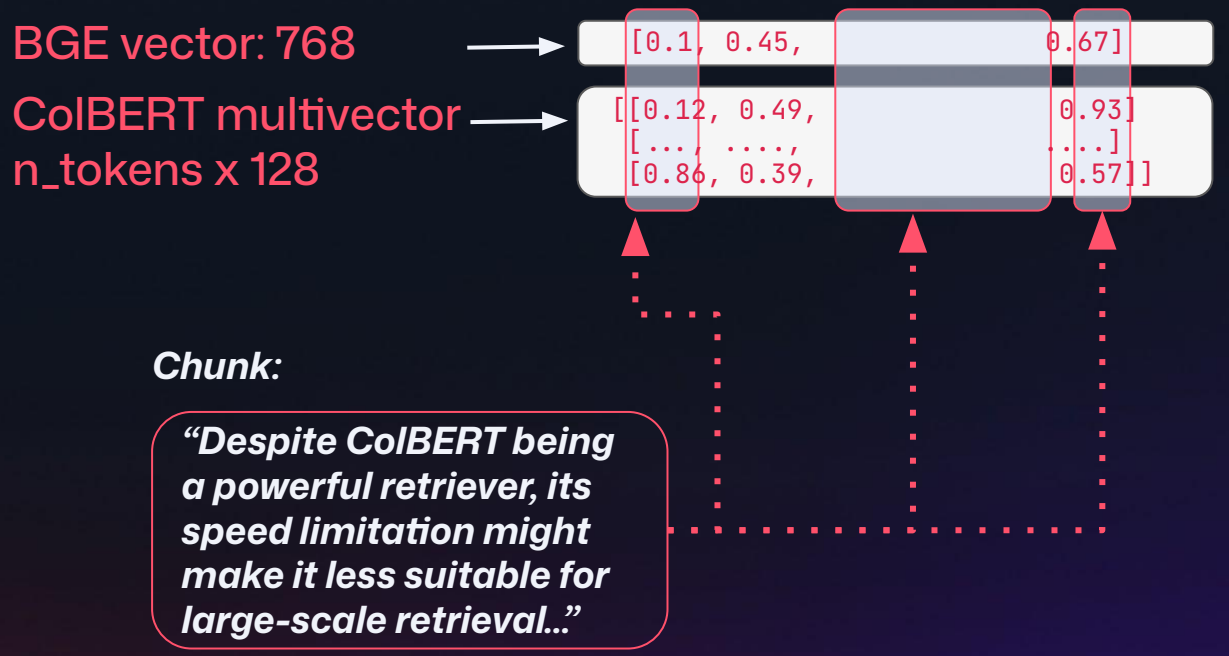




Balancing speed and precision

BGE for scalable candidate retrieval

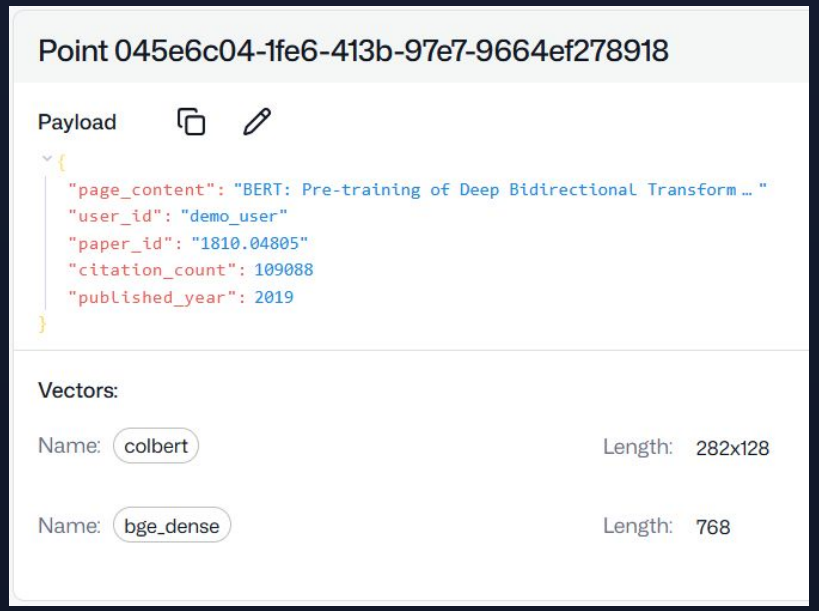
ColBERT for late interaction reranking.



```
dense_model = TextEmbedding("BAAI/bge-base-en-v1.5")
colbert_model = LateInteractionTextEmbedding("colbert-ir/colbertv2.0")

for i, text in enumerate(texts):
    points.append(models.PointStruct(
        id=str(uuid.uuid4()),
        vector={
            "bge_dense": dense_embeds[i].tolist(),
            "colbert": colbert_embeds[i].tolist()
        },
        payload={
            "page_content": text,
            "user_id": user_id,
            "paper_id": arxiv_id,
            "citation_count": citation_count,
            "published_year": pub_year
        }
    ))
```

```python
BATCH_SIZE = 25
for i in range(0, len(points), BATCH_SIZE):
    batch = points[i : i + BATCH_SIZE]
    await qdrant_client.upsert(
        collection_name=COLLECTION_NAME,
        points=batch,
        wait=True
    )
```



3. Multi-stage retrieval

Multi-stage Query

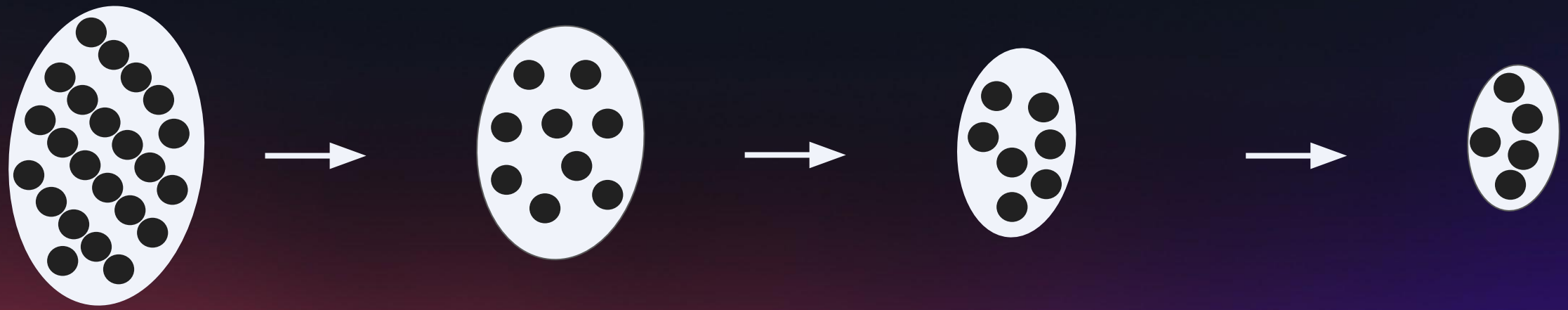
Citations
Recency

Filtering
by user_id

Vector Search
similarity search

Re-ranking
score boosting

Re-ranking
Late interaction
(ColBERT)



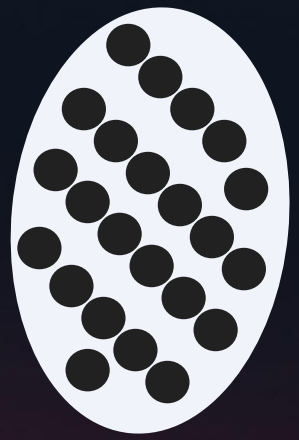
User data

50 chunks

30 chunks

5 chunks

Filtering
by user_id



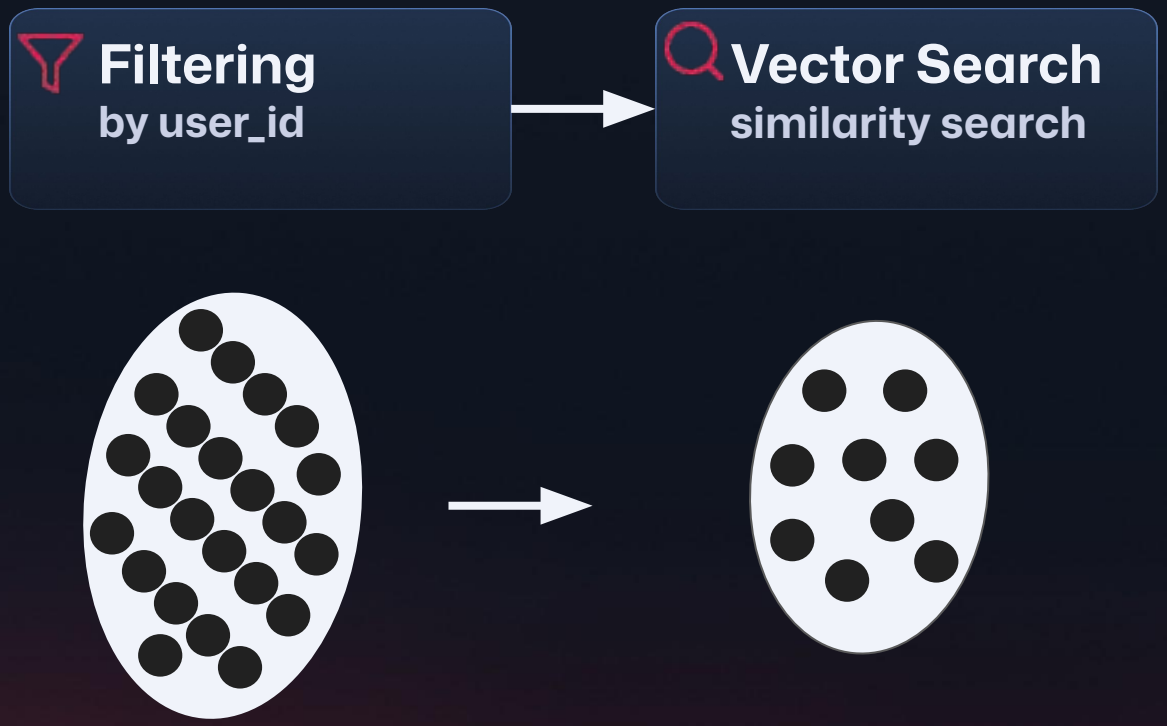
User data

1. Embedding the user query

```
query_dense = list(dense_model.embed([query_text]))[0].tolist()
query_multivector =
list(colbert_model.embed([query_text]))[0].tolist()
```

2. Filtering by user_id

```
response = await qdrant_client.query_points(
        collection_name=COLLECTION_NAME,
        query_filter=models.Filter(
            must=[models.FieldCondition(
                key="user_id",
                match=models.MatchValue(value=user_id)
            )]
        ),
```



3. Semantic Search : **Prefetch**

```
prefetch=[
models.Prefetch(
        query=query_dense,
        using="bge_dense",
        limit=50
       )
],
```

4. Score boosting with FormulaQuery

```
Boosted Score = Similarity Score + 0.1 ln(citations +1) +
                0.05(published year-2000)
```

```
prefetch=[
    models.Prefetch(
        prefetch=[
            models.Prefetch(
                query=query_dense,
                using="bge_dense",
                limit=50,)],
        query=models.FormulaQuery(
            formula=models.SumExpression(
                sum=[
                    "$score",  ← Similarity score from the semantic search step
                    models.MultExpression(mult=[
                        0.1,
                        models.LnExpression(ln=models.SumExpression(sum=[1.0, "citation_count"]))
                    ]),
                    models.MultExpression(mult=[
                        0.05,
                        models.SumExpression(sum=["published_year", -2000.0])])])),
        limit=30,)],
```

Nested prefetches

5. Late interaction re-ranking with colBERT

Citations
Recency

Filtering
by user_id

Vector Search
similarity search

Re-ranking
score boosting

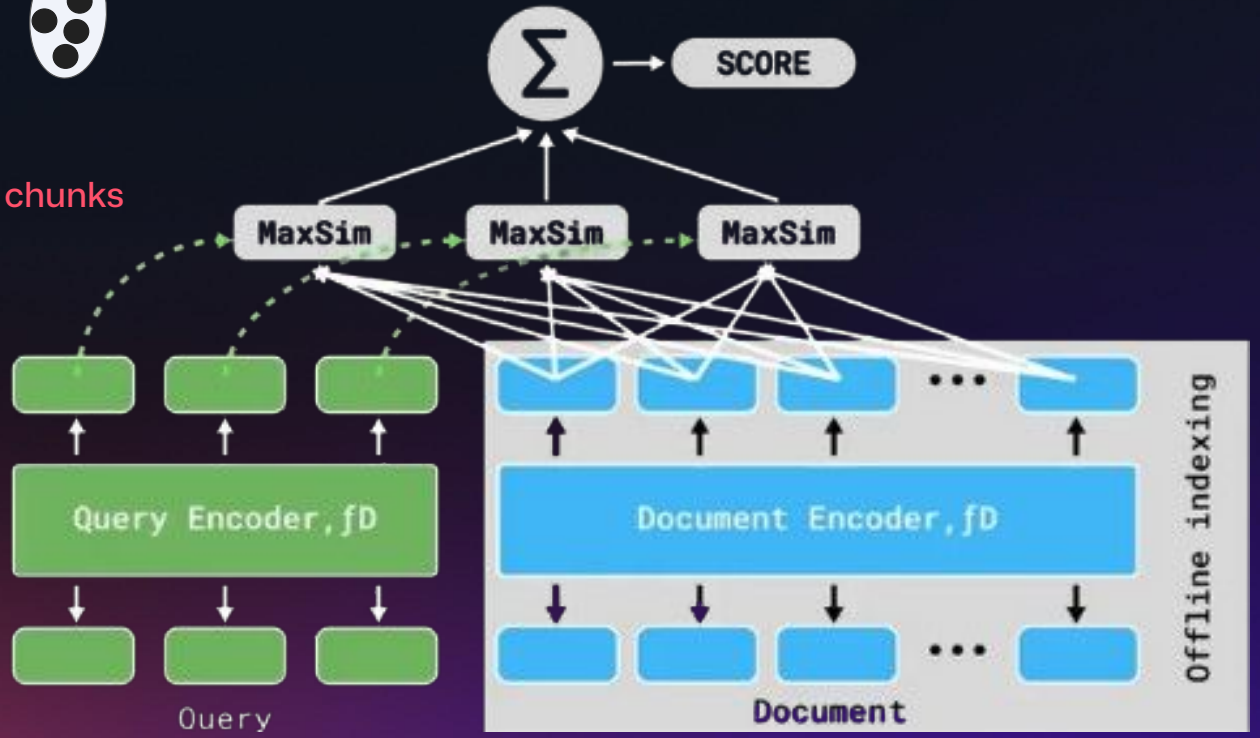
Re-ranking
Late interaction
(ColBERT)

User data

50 chunks

30 chunks

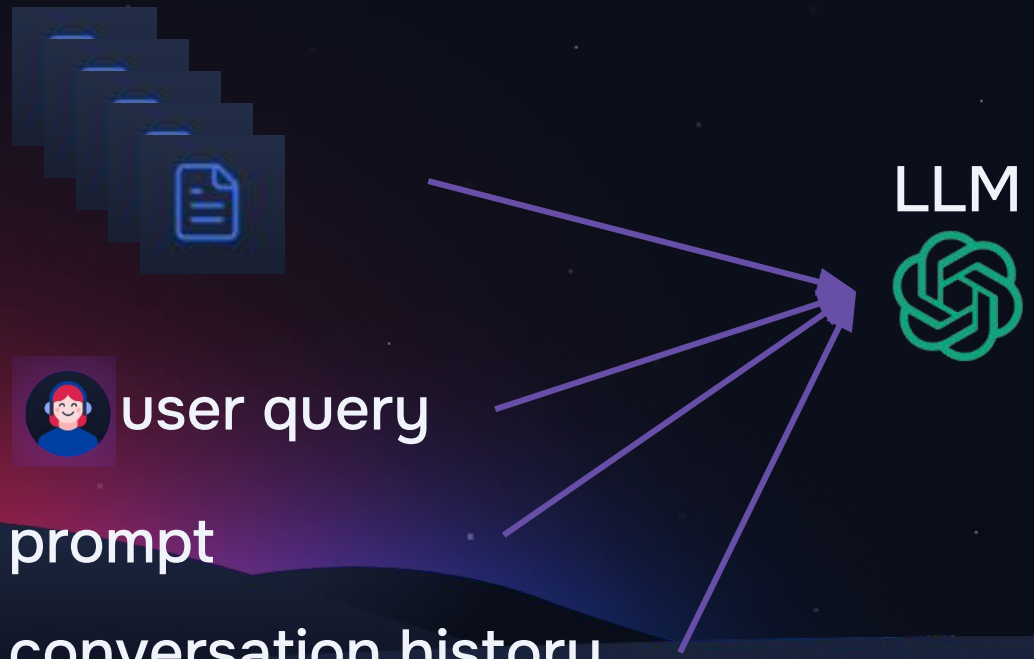
5 chunks

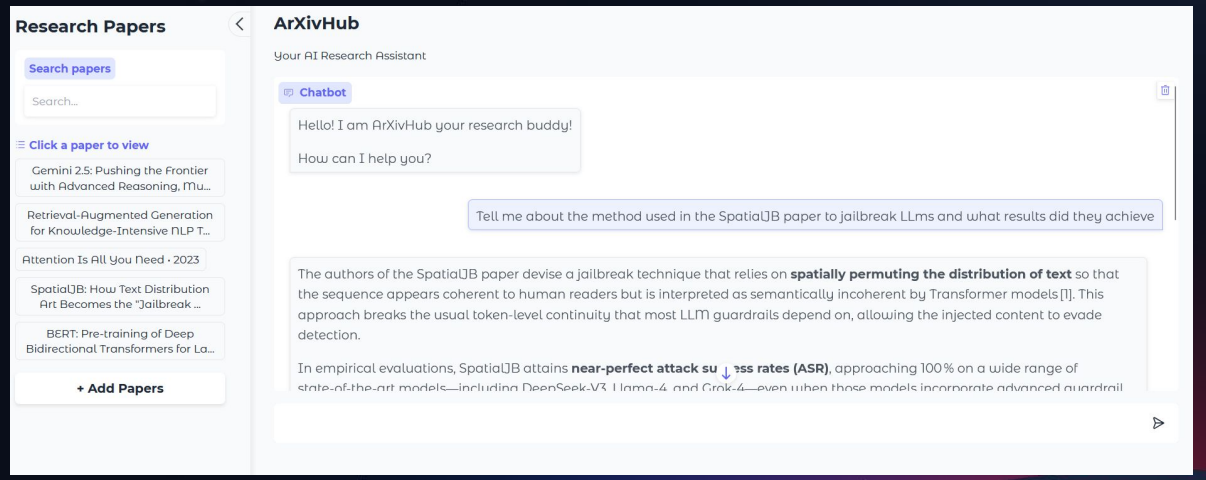


```
prefetch=[models.Prefetch(
                prefetch=[
                    models.Prefetch(
                        query=query_dense,
                        using="bge_dense",
                        limit=50)],
                query=models.FormulaQuery(
                    formula=models.SumExpression(
                        sum=["$score",
                            models.MultExpression(mult=[0.1,
                                models.LnExpression(ln=models.SumExpression(sum=[1.0, "citation_count"]))
                            ]),
                            models.MultExpression(mult=[0.005,
                                models.SumExpression(sum=["published_year", -2000.0])
                            ])])),
                limit=30,)],
        query=query_multivector,
        using="colbert",
        limit=5,
        with_payload=True
    )
    return response.points
```

Putting it all together

- Balance between scalability, latency and quality
- Tune parameter choices to your specific use case





Thank you
Questions ?